

DOKUMENTACJA PROJEKTOWA

Strona tytułowa

Tytuł projektu: LABTRY (Gra typu labirynt)

Zespół projektowy:

- Adam Kurbiel
- Karina Bednarska

Technologie:

- JavaScript (+Canvas API)
- HTML5
- CSS

Czas realizacji:

10.04.2026 - 08.06.2026

1. Ogólny opis projektu

1.1 Cel projektu

LABTRY jest dwuwymiarową grą typu labirynt stworzoną w technologii webowej. Gracz wciela się w bohatera, który budzi się w środku tajemniczego lasu i stopniowo odkrywa historię zniszczonego laboratorium. Celem gry jest przechodzenie kolejnych poziomów, rozwiązywanie zagadek logicznych, zbieranie przedmiotów oraz unikanie przeciwników i pułapek.

1.2 Charakterystyka gry

Projekt wykorzystuje grafikę w stylu pixelart oraz mechanikę eksploracji labiryntów. Każdy poziom zawiera:

- przeszkody,
- przeciwników poruszających się po mapie,
- zagadki logiczne,
- przedmioty (konsumowalne i fabularne)
- elementy fabularne w postaci notatek.

Gra składa się z pięciu poziomów o rosnącym stopniu trudności.

1.3 Architektura projektu

Projekt został podzielony na moduły:

Core

Odpowiada za uruchamianie gry oraz główną pętlę rozgrywki.

Pliki:

- main.js
- game.js

Entities

Zawiera definicje wszystkich obiektów aktywnych występujących w grze.

Pliki:

- player.js
- horizontal_bat.js
- vertical_bat.js
- horizontal_spider.js
- vertical_spider.js
- trap.js

Systems

Systemy wspomagające działanie gry.

Pliki:

- audio.js
- camera.js
- entityHandler.js
- hud.js
- input.js
- itemInfo.js
- renderer.js
- vignette.js

World

Odpowiada za przechowywanie danych map oraz poziomów.

Pliki:

- maps.js

2. Wymagania funkcjonalne

2.1 Sterowanie postacią

System powinien umożliwiać:

- poruszanie postacią za pomocą klawiszy W, A, S, D,
- płynny ruch pomiędzy polami mapy,
- animowanie ruchu postaci,
- blokowanie ruchu przez ściany i przeszkody.

2.2 System poziomów

System powinien:

- ładować kolejne poziomy gry,
- resetować poziom po utracie życia,
- umożliwiać przechodzenie pomiędzy etapami rozgrywki.

2.3 System ekwipunku

Gracz może:

- zbierać przedmioty,
- przechowywać przedmioty w ekwipunku,
- używać wybranych przedmiotów,
- sprawdzić opis i rzadkość przedmiotu.

Przykładowe przedmioty:

- KEY
- BURGER
- MEDKIT

2.4 System zdrowia

System powinien:

- przechowywać aktualne punkty życia gracza,
- odbierać punkty życia po kontakcie z zagrożeniem,
- umożliwiać leczenie przy pomocy przedmiotów.

2.5 System przeciwników

Przeciwnicy powinni:

- poruszać się po wyznaczonej ścieżce,
- zadawać obrażenia graczowi,
- powodować restart poziomu po kontakcie.

Przykłady przeciwników:

- Bat (horizontal/vertical) w zależności od kierunku, porusza się w jedną stronę. Gdy dobieje do ściany/przeszkody, odbija się w przeciwnym kierunku.
- Pułapka podłogowa - Co kilka sekund z podłogi wyskakują kolce. Gdy są schowane, nie stanowią zagrożenia.

2.6 System zagadek

System powinien umożliwiać:

- rozwiązywanie zagadek matematycznych,
- otrzymywanie nagród po poprawnym rozwiązaniu.

2.7 System notatek fabularnych

Gracz może:

- zbierać notatki,
- odczytywać treści fabularne w dowolnym momencie (tylko jeżeli notatka już została zebrana).

2.8 System audio

Gra powinna:

- odtwarzać muzykę tła,
 - odtwarzać efekty dźwiękowe,
 - reagować dźwiękiem na działania gracza.
-

3. Wymagania niefunkcjonalne

3.1 Wydajność

- Gra powinna działać płynnie w nowoczesnych przeglądarkach internetowych.
- Renderowanie powinno odbywać się w czasie rzeczywistym za pomocą pętli gry.

3.2 Użyteczność

- Interfejs powinien być intuicyjny.
- Sterowanie powinno być łatwe do opanowania.
- Informacje o stanie gry powinny być czytelne.

3.3 niezawodność

- Gra nie powinna powodować utraty postępu w obrębie poziomu.
- Obsługa kolizji powinna działać poprawnie.

3.4 Przenośność

- Aplikacja powinna działać na systemach Windows, Linux i macOS.
- Wymagana jest jedynie przeglądarka internetowa obsługująca JavaScript.

3.5 Rozszerzalność

Architektura projektu powinna umożliwiać:

- dodawanie nowych poziomów,
- dodawanie nowych przeciwników,
- dodawanie nowych przedmiotów,
- rozbudowę fabuły.

3.6 Estetyka

- Grafika powinna zachowywać styl Pixel Art.
 - Elementy interfejsu powinny być spójne wizualnie.
-

4. Dokumentacja kodu

4.1 Plik index.html

Główny plik aplikacji.

Odpowiada za:

- utworzenie obszarów Canvas,
- wyświetlenie menu startowego,
- załadowanie głównego modułu JavaScript.

4.2 Moduł main.js

Odpowiada za:

- inicjalizację gry,
- tworzenie obiektów systemowych,
- ładowanie czcionek,
- uruchamianie rozgrywki,
- obsługę zdarzeń myszy.

4.3 Moduł game.js

Odpowiada za:

- główną pętlę gry,
- aktualizację stanu świata,
- obsługę zagadek,
- przechodzenie między poziomami,
- restart poziomów,
- renderowanie elementów gry.

4.4 Klasa Player

Opisuje gracza.

Wybrane właściwości:

- health (punkty życia)
- inventory (ekwipunek)
- x (koordynaty poziome)
- y (koordynaty pionowe)

- paused (zatrzymane poruszanie się)
- currentNote (aktualnie czytana notatka)

Wybrane metody:

- **resetPosition()** - Ustawia pozycję startową gracza.
- **update()** - Aktualizuje stan gracza podczas rozgrywki.
- **move()** - Obsługuje ruch postaci oraz interakcję z otoczeniem.
- **deleteItem()** - Usuwa przedmiot z ekwipunku.

4.5 System EntityHandler

Odpowiada za:

- przechowywanie wszystkich bytów,
- aktualizację przeciwników.

4.6 System Camera

Odpowiada za:

- śledzenie pozycji gracza w płynny sposób,
- wyświetlanie odpowiedniego fragmentu mapy.

4.7 System Audio

Odpowiada za:

- odtwarzanie muzyki,
- odtwarzanie efektów dźwiękowych.

4.8 System HUD

Odpowiada za:

- wyświetlanie ekwipunku i używanie przedmiotów w nim,
- wyświetlanie statystyk gracza,
- wyświetlanie aktualnych efektów

4.9 Moduł maps

Zawiera:

- zdefiniowane poziomy,
- treści notatek fabularnych,
- metody wczytujące aktualny poziom oraz konwertujące zdefiniowane poziomy na tablice.

Mapy zapisane są jako tablice znaków, gdzie każdy symbol reprezentuje określony element gry.

Przykłady:

- S - punkt startowy
- E - wyjście z poziomu
- D - drzwi
- K - klucz
- N - notatka
- C - skrzynia logiczna
- T - pułapka
- # - ściana

4.10 System renderer

Renderer odpowiada za rysowanie obiektów na głównym canvasie (game), czyli przeciwników oraz mapę.

system ten może obsługiwać renderowanie tekstur, a dla obiektów które nie mają takowych zdefiniowanych, może renderować kolor.

gracz jak i przeciwnicy dynamiczni (poruszający się w świecie gry) mają być renderowani płynnie, czyli między przejściami pomiędzy kafelkami ma następować płynne przejście.

4.11 Zależności pomiędzy modułami

W projekcie mają występować następujące zależności między modułami (po kolei):

1. main.js
2. game.js
3. Player, Map, Camera, EntityHandler
4. HUD, Audio, Renderer

Takie rozdzielenie odpowiedzialności zwiększa czytelność kodu oraz ułatwia pracę nad jego rozwojem.

Podsumowanie

Projekt LABTRY jest przeglądarkową grą labiryntową wykorzystującą JavaScript, HTML oraz CSS. Aplikacja implementuje system poziomów, przeciwników, zagadek logicznych, ekwipunku oraz elementów fabularnych. Modułarna budowa kodu pozwala na łatwe rozszerzanie funkcjonalności oraz dalszy rozwój projektu.